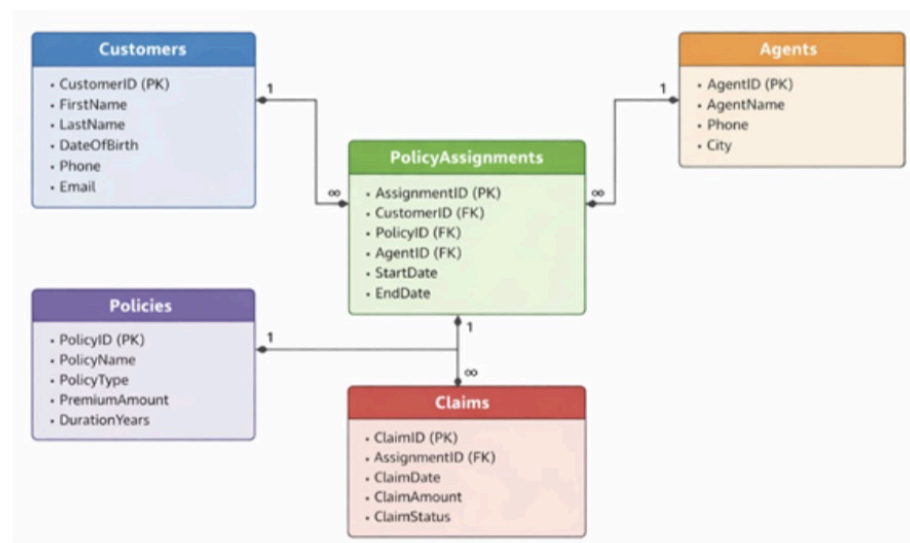


Module 4.4 practical project assignment

BY SHAAZ



Insurance DB creation:-

Bold chars are for important topics:-

```
CREATE TABLE Customers (
    CustomerID INT IDENTITY PRIMARY KEY,
    FirstName VARCHAR(20) NOT NULL,
    LastName VARCHAR(20) NOT NULL,
    DateOfBirth DATE NOT NULL,
    Phone VARCHAR(15) NOT NULL,
    Email VARCHAR(50) NOT NULL UNIQUE
);
```

```
CREATE TABLE Agents (
    AgentId INT IDENTITY PRIMARY KEY,
    AgentName VARCHAR(20) NOT NULL,
```

```
    Phone VARCHAR(15) NOT NULL,  
    City VARCHAR(20) NOT NULL  
);
```

```
CREATE TABLE Policies (  
    PolicyId INT IDENTITY PRIMARY KEY,  
    PolicyName VARCHAR(20) NOT NULL,  
    PolicyType VARCHAR(20) NOT NULL,  
    PremiumAmount MONEY NOT NULL,  
    DurationYears INT NOT NULL  
);
```

```
CREATE TABLE PolicyAssignments (  
    AssignmentID INT IDENTITY PRIMARY KEY,  
    CustomerID INT NOT NULL,  
    PolicyId INT NOT NULL,  
    AgentId INT NOT NULL,  
    StartDate DATE NOT NULL,  
    EndDate DATE NOT NULL,  
  
    CONSTRAINT FK_PA_Customers  
        FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID),  
  
    CONSTRAINT FK_PA_Policies  
        FOREIGN KEY (PolicyId) REFERENCES Policies (PolicyId),  
  
    CONSTRAINT FK_PA_Agents  
        FOREIGN KEY (AgentId) REFERENCES Agents (AgentId)  
);
```

```
CREATE TABLE Claims (  
    ClaimsID INT IDENTITY PRIMARY KEY,  
    AssignmentID INT NOT NULL,  
    ClaimDate DATE NOT NULL,  
    ClaimAmount MONEY NOT NULL,  
    ClaimStatus VARCHAR(20) NOT NULL,  
  
    CONSTRAINT FK_Claims_Assignments  
        FOREIGN KEY (AssignmentID) REFERENCES PolicyAssignments (AssignmentID),  
  
);
```

Insertions:-

```
INSERT INTO Customers (FirstName,LastName,DateOfBirth,Phone,Email)
values('Amit','Sharma','1990-05-10','9876543210','amit@gmail.com');
INSERT INTO Customers (FirstName,LastName,DateOfBirth,Phone,Email)
values('Shaaz','Hussain','1990-05-11','9876543230','shaaz@gmail.com');

INSERT INTO Customers (FirstName,LastName,DateOfBirth,Phone,Email)
values('Bharath','Simha','1990-05-13','9876543240','Bharath@gmail.com');

INSERT INTO Policies (PolicyName,PolicyType,PremiumAmount,DurationYears)
VALUES ('Health Secure','Health',12000.00,1)
INSERT INTO Policies (PolicyName,PolicyType,PremiumAmount,DurationYears)
VALUES ('Life Shield','Life',25000.00,1)
INSERT INTO Policies (PolicyName,PolicyType,PremiumAmount,DurationYears)
VALUES ('Motor Protect','Motor',8000.00,1)

select * from Customers;
select * from Agents;
select * from Policies;

INSERT INTO Agents ( AgentName, Phone, City)
VALUES
( 'Ravi Kumar', '9123456789', 'Hyderabad'),
( 'Anita Verma', '9234567890', 'Bangalore');

INSERT INTO PolicyAssignments
( CustomerID, PolicyId, AgentId, StartDate, EndDate)
VALUES
( 1,1,1,'2024-01-01', '2024-12-31'),
( 2,3,2, '2024-02-01', '2025-01-31'),
(3,2,1,'2024-03-01', '2025-02-28');

SELECT * from PolicyAssignments;
SELECT * from Claims;
```

```

INSERT INTO Claims
( ClaimsID,AssignmentID, ClaimDate, ClaimAmount, ClaimStatus)
VALUES
( 1,3, '2024-06-15', 5000.00, 'Approved'),
( 2,4, '2024-07-10', 10000.00, 'Pending'),
(3,5, '2024-08-05', 3000.00, 'Rejected');

select * from Claims;

```

CRUD OPERATIONS:-

SELECT WILD CARD -> IS CALLED

FROM → DATA SOURCE NAME, SOURCE CAN BE MULTIPLE

WHERE → CONDITION//FILTER//BOOLEAN EXPRESSION//→ IT IS USED USING
BOOLEAN OPERATORS COLUMN OP VALUE

ORDER BY COLUMN NAME SORT CRITERIA→BY DEFAULT ASCENDING(ASC,DESC)

ORDER BY CAN HAVE MULTIPLE COLUMNS

GROUP BY GROUP CRITERIA →CAN BE PUT UPON AGGREGATION FUNCTIONS

HAVING CONDITION→ **USED TO APPLY CONDITION OVER GROUP**
ALWAYS ASSOCIATED WITH GROUP

DISTINCT —> ONLY WORKS ON RESULT NOT ON SOME COLUMN

TOP AND PERCENT

OFFSET AND FETCH→USED WITH ORDER BY TO SKIP OR LIMIT THE ROWS

LIKE %,_,[],^

CALCULATED FIELDS:-

```
SELECT FirstName+'('+Email+')' from Customers order by FirstName;
```

4. Select Commands

```
SELECT * FROM Customers;
```

```
SELECT * FROM Agents;
```

```
SELECT * FROM Policies;
```

```
SELECT * FROM PolicyAssignments;
```

```
SELECT * FROM Claims;
```

5. SELECT COMMANDS WITH CONDITIONS

View all records of PolicyAssignment table with CustomerId, PolicyId, StartDate and EndDate columns only.

```
SELECT CustomerID,PolicyId,StartDate,EndDate from PolicyAssignments;
```

Display all policies of Health type.

```
SELECT CustomerID,PolicyId,StartDate,EndDate from PolicyAssignments;
```

Display policies having premium amount more than 10000 and DurationYears is 1.

```
select * from Policies where PremiumAmount>10000 and  
DurationYears=1;
```

Display unique city names from where agents belong to.

```
SELECT distinct City from Agents;
```

List policies of type Life, Health, Motor use OR clause.

```
select * from Policies where PolicyType='Term Life Shield'  
or PolicyType='Health' or PolicyType='Motor';
```

List policies of type Life, Health, Motor use IN operator.

```
select * from Policies where PolicyType in ('Term Life  
Shield', 'Health' , 'Motor');
```

Display list of customers born after January 1 st , 2001 and before December 31 st , 2020 using >= and <= operators.

```
select * from Customers where DateOfBirth>'2001-01-01' and  
DateOfBirth<'2020-12-31';
```

Display list of customers born after January 1 st , 2001 and before December 31 st , 2020 using between operator.

```
select * from Customers where DateOfBirth between  
'2001-01-01' and '2020-12-31';
```

Display claims data where claim status is Rejected.

```
select * from Claims where ClaimStatus='Rejected';
```

Display records of Agents who stay in a city whose second letter is 'a'.

```
select * from Agents where City like '_a%';
```

Display highest and lowest claimAmount from Claims table.

```
select max(ClaimAmount) as  
Highest_Claim_Amount,min(ClaimAmount) as  
Lowest_Claim_Amount from Claims;
```

Display latest claim record.

```
select max(ClaimDate) as Latest_claim from Claims;
```

Increase premium amount to 10% for all health insurance policies.

```
select PremiumAmount*1.1 as Increased_Premium from  
Policies where PolicyType='Health';  
update Policies set PremiumAmount=1.1*PremiumAmount where  
PolicyType='Health';
```

Delete the record of PolicyAssignments whose EndDate is before today's date.

```
select * from PolicyAssignments where EndDate<getdate();  
delete from PolicyAssignments where EndDate<getdate();
```

Display no of claims rejected.

```
select count(*) as Number_of_rejected from Claims where  
ClaimStatus='Rejected';
```

Display PolicyId, PolicyName, PremiumAmount along with computed fields not in table à 6% LocalTaxes, PremiumAmountWithTax and MonthlyPremiumAmount considering PremiumAmount is Annual.

6. UPDATE QUERIES: -

Increase premium amount to 10% for all health insurance policies.

```
update Policies set PremiumAmount=1.1*PremiumAmount where  
PolicyType='Health';
```

6. DELETE QUERIES:-

Delete the record of PolicyAssignments whose EndDate is before today's date.

```
select * from PolicyAssignments where EndDate<getdate();  
delete from PolicyAssignments where EndDate<getdate();
```

7. Alter Table Queries

Write a command to add Address and City Columns in the Customers table.

```
alter table Customers add Address VARCHAR(20), City  
VARCHAR(20);  
select * from Customers;
```

Write a command to add a new column named DevOfId (DevelopmentOfficerId) in an existing Agents table.

```
alter table Agents add DevOfId VARCHAR(20);  
select * from Agents;
```


8. Join, Group By & Having Queries

List all Policies for a CustomerId 5.

```
SELECT * from Policies p
join PolicyAssignments p1
on p1.PolicyId=p.PolicyId
where CustomerId=3
```

View all customers with their policies.

```
SELECT c.*,p.* from Policies p
join PolicyAssignments p1
on p1.PolicyId=p.PolicyId
join Customers c
on p1.CustomerId=c.CustomerId
```

View claims with customer name.

```
SELECT c.*,cl.FirstName from Claims c
join PolicyAssignments p
on p.AssignmentId=c.AssignmentId
join Customers cl
on cl.CustomerId=p.CustomerId;
```

Display FirstName, PolicyName, AgentName, StartDate and EndDate from their respective tables.

```
SELECT
c.FirstName,p.PolicyName,a.AgentName,p1.StartDate,p1.EndDate
from PolicyAssignments p1
join Customers c
on p1.CustomerId=c.CustomerId
join Policies p
on p.PolicyId=p1.PolicyId
join Agents a
```

```
on a.AgentId=p1.AgentId;
```

Display claims report with FirstName, PolicyName, ClaimAmount, ClaimStatus, and ClaimDate from their respective tables.

```
SELECT
c.FirstName,p.PolicyName,c1.ClaimAmount,c1.ClaimStatus,c1.
ClaimDate
from Claims c1
join PolicyAssignments p1
on p1.AssignmentId=c1.AssignmentId
join Customers c
on c.CustomerId=p1.CustomerId
join Policies p
on p.PolicyId=p1.PolicyId
```

Display records of Customers with or without Policies.

```
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    p.PolicyName,
    pa.StartDate,
    pa.EndDate
FROM Customers c
LEFT JOIN PolicyAssignments pa
ON c.CustomerID = pa.CustomerID
LEFT JOIN Policies p
ON pa.PolicyId = p.PolicyId;
```

Display all Customers with NO Claims.

```
SELECT DISTINCT
    c.CustomerID,
    c.FirstName,
    c.LastName
```

```

FROM Customers c
LEFT JOIN PolicyAssignments pa
    ON c.CustomerID = pa.CustomerID
LEFT JOIN Claims cl
    ON pa.AssignmentID = cl.AssignmentID
WHERE cl.ClaimsID IS NULL;

```

Show CustomerName with Total Claim Amount per Customer.

```

SELECT
    c.FirstName AS CustomerName,
    SUM(cl.ClaimAmount) AS TotalClaimAmount
FROM Customers c
JOIN PolicyAssignments pa
    ON c.CustomerID = pa.CustomerID
JOIN Claims cl
    ON pa.AssignmentID = cl.AssignmentID
GROUP BY
    c.FirstName,
    c.LastName;

```

9. FUNCTIONS Queries:-

```

-- 1:-
select FirstName+' '+LastName as FullName from Customers;

-- 2:-
select FirstName from Customers where FirstName like 'A%';

-- 3:-
select upper(LastName) from Customers;

-- 4:-

```

```

select * from Customers
where len(LastName)>6;

-- 5:-
select SUBSTRING(FirstName,0,4) from Customers;

-- 6:-
UPDATE Policies set PolicyName=replace(PolicyName,'Term Life Shield','Term Life')
where PolicyName like '%Life%';
select * from Policies;

-- 7:-
select FirstName from Customers where FirstName like '%sha%';

-- 8:-
select RTRIM(ltrim(FirstName)) from Customers;

-- 9:-
select PolicyType,len(PolicyType) as length_of_char from Policies ;

-- 10:-
select right(PolicyType,4) from Policies;

-- 11:-
select ROUND(PremiumAmount,1) from Policies;

-- 12:-
select ROUND(PremiumAmount,1) from Policies where round(PremiumAmount,1)>5000;

-- 13:-
select abs(ClaimAmount) from Claims;

-- 14:-
select 0.1*PremiumAmount as tax from Policies;

-- 15:-
select ROUND(PremiumAmount,2) from Policies;

-- 16:-
select sqrt(PremiumAmount) from Policies;

-- 17:-
select CEILING(ClaimAmount) from Claims;

-- 18:-

```

```

select FLOOR(PremiumAmount) from Policies;

-- 19:-
select PremiumAmount+(PremiumAmount*0.18) as PremiumAmount_and_GST from Policies;

-- 20:-
select ClaimAmount from Claims where ClaimAmount%2=0;

-- 21:-
SELECT GETDATE() as curr_date;

```

10.Date and Time Queries:-

```

SELECT
    GETDATE() AS Current_DateTime,
    CAST(GETDATE() AS DATE) AS Current_Date,
    CAST(GETDATE() AS TIME) AS Current_Time;

SELECT DATENAME(WEEKDAY, GETDATE()) AS DayName;

SELECT CAST(GETDATE() AS DATE) AS CurrentDate;

SELECT CAST(GETDATE() AS TIME) AS CurrentTime;

SELECT *
FROM Customers
WHERE MONTH(DateOfBirth) = MONTH(GETDATE())
    AND DAY(DateOfBirth) = DAY(GETDATE());

SELECT *
FROM Claims
WHERE ClaimDate >= DATEADD(DAY, -30, GETDATE());

```

```
SELECT
    AssignmentID,
    EndDate,
    DATEDIFF(DAY, GETDATE(), EndDate) AS DaysRemaining
FROM PolicyAssignments;
```

11.SubQueries:-

```
SELECT *
FROM Customers
WHERE CustomerID IN (
    SELECT CustomerID
    FROM PolicyAssignments
);
```

```
SELECT *
FROM Customers
WHERE CustomerID NOT IN (
    SELECT CustomerID
    FROM PolicyAssignments
);
```

```
SELECT *
FROM Policies
WHERE PremiumAmount > (
    SELECT AVG(PremiumAmount)
    FROM Policies
);
```

```
SELECT *
FROM Policies
WHERE PremiumAmount = (
```

```
        SELECT MAX(PremiumAmount)
        FROM Policies
    );
```

```
SELECT *
FROM Customers
WHERE CustomerID IN (
    SELECT pa.CustomerID
    FROM PolicyAssignments pa
    WHERE pa.AssignmentID IN (
        SELECT AssignmentID
        FROM Claims
    )
);
```

```
SELECT *
FROM Customers
WHERE CustomerID NOT IN (
    SELECT pa.CustomerID
    FROM PolicyAssignments pa
    WHERE pa.AssignmentID IN (
        SELECT AssignmentID
        FROM Claims
    )
);
```

```
SELECT *
FROM Policies
WHERE PolicyId IN (
    SELECT PolicyId
    FROM PolicyAssignments
    GROUP BY PolicyId
    HAVING COUNT(*) > 1
);
```

```
SELECT *
FROM Agents
WHERE AgentId IN (
    SELECT AgentId
    FROM PolicyAssignments
);
```

```
SELECT *
FROM Agents
WHERE AgentId NOT IN (
```

```

        SELECT AgentId
        FROM PolicyAssignments
    );

SELECT *
FROM Customers
WHERE CustomerID IN (
    SELECT pa.CustomerID
    FROM PolicyAssignments pa
    JOIN Claims cl
    ON pa.AssignmentID = cl.AssignmentID
    GROUP BY pa.CustomerID
    HAVING SUM(cl.ClaimAmount) > 50000);

```

12.CASE ELSE:-

```

SELECT
    PolicyId,
    PolicyName,
    PremiumAmount,
    CASE
        WHEN PremiumAmount < 10000 THEN 'Low Premium'
        WHEN PremiumAmount BETWEEN 10000 AND 20000 THEN 'Medium Premium'
        ELSE 'High Premium'
    END AS PremiumCategory
FROM Policies;

```

```

SELECT
    ClaimsID,
    ClaimAmount,
    ClaimStatus,
    CASE
        WHEN ClaimStatus = 'Approved' THEN 'Claim Settled'
        WHEN ClaimStatus = 'Rejected' THEN 'Claim Denied'
        ELSE 'Under Review'
    END AS ClaimRemark
FROM Claims;

```

```

SELECT
    a.AgentName,
    COUNT(pa.PolicyId) AS PolicyCount,
    CASE

```



```

        WHEN COUNT(pa.PolicyId) >= 5 THEN 'Top Performer'
        WHEN COUNT(pa.PolicyId) BETWEEN 2 AND 4 THEN 'Average Performer'
        ELSE 'Low Performer'
    END AS Performance
FROM Agents a
LEFT JOIN PolicyAssignments pa
ON a.AgentId = pa.AgentId
GROUP BY a.AgentName;

```

13.GROUP BY ROLLUP:-

```

SELECT
    c.FirstName,
    SUM(cl.ClaimAmount) AS TotalClaimAmount
FROM Customers c
JOIN PolicyAssignments pa ON c.CustomerID = pa.CustomerID
JOIN Claims cl ON pa.AssignmentID = cl.AssignmentID
GROUP BY ROLLUP (c.FirstName);

```

```

SELECT
    a.AgentName,
    COUNT(pa.PolicyId) AS PolicyCount
FROM Agents a
LEFT JOIN PolicyAssignments pa
ON a.AgentId = pa.AgentId
GROUP BY ROLLUP (a.AgentName);

```

14.GROUP BY CUBE:-

```

SELECT
    c.FirstName,
    p.PolicyName,
    SUM(cl.ClaimAmount) AS TotalClaimAmount
FROM Claims cl
JOIN PolicyAssignments pa ON cl.AssignmentID = pa.AssignmentID
JOIN Customers c ON pa.CustomerID = c.CustomerID

```

```
JOIN Policies p ON pa.PolicyId = p.PolicyId
GROUP BY CUBE (c.FirstName, p.PolicyName);
```

15.GROUP BY SETS:-

```
SELECT
    c.FirstName,
    p.PolicyName,
    SUM(cl.ClaimAmount) AS TotalClaimAmount
FROM Claims cl
JOIN PolicyAssignments pa ON cl.AssignmentID = pa.AssignmentID
JOIN Customers c ON pa.CustomerID = c.CustomerID
JOIN Policies p ON pa.PolicyId = p.PolicyId
GROUP BY GROUPING SETS (
    (c.FirstName),
    (p.PolicyName),
    ()
);
```

```
SELECT
    a.AgentName,
    a.City,
    COUNT(pa.PolicyId) AS PolicyCount
FROM Agents a
LEFT JOIN PolicyAssignments pa
ON a.AgentId = pa.AgentId
GROUP BY GROUPING SETS (
    (a.AgentName),
    (a.City),
    ()
);
```

IMPORTANT POINTS:-

- Alter table for adding constraints like unique
- 2 types of constraints description
- We can
- `CONSTRAINT CHK_ClaimStatus`
- `CHECK (ClaimStatus IN ('Pending', 'Approved', 'Rejected'))`
this if we want in the last table
- Can use CHECK for keeping business rule;
- Indexing in sql internal system of rdbms theory
- Trim functions we can only trim whitespaces
- Dates, string, number are important read functions
- DatePart() imp
- Read System Functions
- Read Aggregate Functions
- As grouping we can use aggregate functions cant display group+scalar
On delete cascade
On update cascade